P(F) Shape Matching

Phase 5: SAPO Training Plan

Differentiable Surgical Debulking Simulation + Policy Learning

CUSA Debulking: From Differentiable Sim to Real Robot

Target: CoRL 2026 (~Jun 2026)

Method: SAPO (Soft Analytic Policy Optimization, ICLR 2025)

Platform: NVIDIA Warp + PyTorch

1. Project Status

Phase 1-4 completed. Warp-based P(F) solver is functional and differentiable.

Phase	Content	Status
Phase 1	Mesh loading + Warp array init	DONE
Phase 2	Core Shape Matching (SVD)	DONE
Phase 3	P(F) Damage + Soft Removal	DONE
Phase 4	Autodiff Verification	DONE
Phase 5	SAPO Training Env + Policy	IN PROGRESS
Phase 6	Paper Experiments	TODO

Phase 4 verified: 5-step gradient flows through knife_pos -> edge_damage -> particle_damage -> alpha -> loss. Finite difference matches analytic direction.

2. Training Method: SAPO

SAPO (ICLR 2025) = short-horizon analytic gradient + actor-critic RL + max entropy.

2.1 SAPO vs Alternatives

Dimension	SAPO	SHAC	PPO
Gradient source	Diff sim (analytic)	Diff sim (analytic)	Reward sampling
Backprop steps	10-50	10-50	0
Sample efficiency	17x vs PPO	17x vs PPO	1x (baseline)
Deformable support	Yes (Rewarped)	Rigid only	Any sim
Stability	High (entropy)	Medium	Very high
Exploration	Max-entropy auto	None (collapse risk)	Clip ratio

2.2 SAPO Algorithm

For each epoch:

1. obs = env.initialize_trajectory() -- reset, cut gradient
2. Record H=32 steps inside wp.Tape():
    action ~ pi_theta(obs) via reparameterization
    obs, rew, done = env.step(action) -- differentiable physics
3. tape.backward() -- analytic gradient through sim
4. Update actor with policy gradient (analytic, not REINFORCE)
5. Update critic via TD(lambda) MSE
6. Update entropy temperature alpha

3. System Architecture

```
Action (3D velocity) -> apply_action_kernel -> knife_pos_new
  -> solver._substep() x sim_substeps
    -> predict_kernel (gravity + damping)
    -> collision_kernel (floor + knife sphere SDF)
    -> cluster_solve_kernel (SVD shape matching)
```

```
  -> apply_kernel (stiffness correction)
  -> update_edge_damage_kernel (P(F) sigmoid)
  -> update_particle_damage_kernel (atomic_max)
  -> update_soft_removal_kernel (sigmoid alpha)
-> compute_obs kernels -> obs_buf (7D)
-> compute_reward_kernel -> rew_buf
```

All operations inside wp.Tape(). Gradient flows: reward -> alpha -> damage -> knife_pos -> action -> policy parameters.

4. Observation / Action / Reward

4.1 Observation (7D)

Dim	Name	Source	Differentiable
0-2	ee_pos (x,y,z)	knife_pos array	Yes
3	local_pf_mean	Mean damage near knife (3x radius)	Yes
4	volume_removed	Sum(1-alpha) for non-fixed particles	Yes
5	nerve_damage	Sum(damage) on nerve-labeled particles	Yes
6	dist_to_nerve	Min distance to nerve particle	No (info only)

4.2 Action (3D)

End-effector velocity (dx, dy, dz). Applied via: knife_pos_new = knife_pos + action * action_scale. Bounded by tanh in actor network.

4.3 Reward

r = +delta_volume_removed - lambda * delta_nerve_damage - mu * ||action||^2 + bonus * sigmoid(V - target)

Default: lambda=5.0, mu=0.01, bonus=10.0, target=50 particles

5. SAPO Hyperparameters

Parameter	Value	Note
horizon_len	32	SAPO/SHAC default; Phase 4 verified 5 steps
sim_substeps	4	Physics substeps per env.step()
actor_lr	2e-3	AdamW optimizer
critic_lr	5e-4	AdamW optimizer
beta1	0.7	Lower than default 0.9 for analytic gradients
gamma	0.99	Discount factor
lambda (TD)	0.95	TD(lambda) for critic targets
max_grad_norm	0.5	Gradient clipping (SAPO uses 0.5)
critic_iterations	16	Critic updates per epoch
no_target_critic	True	SAPO key: no target network
init_alpha	1.0	Entropy temperature initial value
target_entropy	-1.5	-dim(A)/2 = -3/2
hidden_dim	64	Actor/critic MLP hidden size
activation	SiLU	Smooth activation (not ReLU/ELU)
normalization	LayerNorm	On each hidden layer

6. Key Implementation Details

6.1 knife_pos Differentiability (Critical Fix)

knife_pos changed from scalar wp.vec3 to wp.array(dtype=wp.vec3, shape=(1,), requires_grad=True). This enables gradient flow through collision_kernel and update_edge_damage_kernel. All kernel signatures updated to read knife_pos[0] from array.

6.2 Particle Damage Atomic Max

update_particle_damage_kernel uses int-scaled wp.atomic_max (3-pass: float->int, atomic_max, int->float). This is NOT differentiable, but gradient still flows through edge_damage -> soft_removal sigmoid.

6.3 PyTorch-Warp Gradient Bridge

tape.backward() computes d(reward)/d(action) for each timestep. These analytic gradients are transferred to PyTorch via numpy and used as the policy gradient signal (replacing REINFORCE's score function).

7. File Structure

File	Description
warp_sim/envs/debulk_env.py	SAPO-compatible Gym environment
warp_sim/kernels/obs_reward.py	Observation + reward Warp kernels
warp_sim/train_sapo.py	SAPO training entry point

warp_sim/config/debu1k_sapo.yaml	Training hyperparameters
warp_sim/test_env.py	Environment smoke test
warp_sim/test_gradient_32step.py	32-step gradient stability test
pf_shape_matching_warp/sim/so1ver.py	P(F) so1ver (knife_pos refactored)
pf_shape_matching_warp/sim/kerne1s.py	PBD kerne1s (knife_pos array)
pf_shape_matching_warp/sim/pf_damage.py	Damage kerne1s (knife_pos array)

8. Time1ine

Week	Task	De1iverable
1-2	Debu1kEnv + gradient verification	test_env + test_gradient_32step pass
3-4	SAPO training + hyperparameter tuning	Po1icy 1earns vo1ume up + nerve down
5	Ab1ation (SAPO vs PPO vs SAC)	Samp1e efficiency comparison
6	Visualization + paper figures	Po1yscope trajectory + Pareto curves
7-8	Paper writing -> CoRL 2026	Comp1ete draft

9. Risks and Mitigations

Risk	Impact	Mitigation
32-step gradient exp1osion	Training fai1s	Reduce to 16 steps / add checkpointing
atomic_max breaks gradient	Parti1 gradient	Edge damage is differentiab1e; a1pha sigmoid preserves f1ow
PyTorch-Warp bridge	Comp1ex interop	Use action.grad from Warp tape direct1y
GPU memory (32 steps x 20k)	OOM	Reduce partic1es (5k) / reduce horizon
Cutting interva1 too sparse	No gradient signa1	Set cutting_interva1=1

10. Success Criteria

1. test_env.py passes (obs shape, step 1oop, reset)
2. test_gradient_32step.py passes (grad_norm < 1e4, no NaN/Inf)
3. Training on cube_dense.msh: reward increases over 100 epochs
4. Po1icy achieves vo1ume_removed > 30, nerve_damage < 5
5. SAPO achieves 10x+ samp1e efficiency vs PPO base1ine